

TAGORE COLLEGE OF ARTS AND SCIENCE

**(Affiliated to University of Madras)
Chrompet, Chennai-44**



DEPARTMENT OF COMPUTER SCIENCE

RELATIONAL DATABASE MANAGEMENT SYSTEM

NAME :

CLASS :

REG NO :

TAGORE COLLEGE OF ARTS AND SCIENCE

(Affiliated to the University of Madras)

Chrompet, Chennai – 600 044



DEPARTMENT OF COMPUTER SCIENCE

Certified to be the Bonafide Record of Practical work done by Mr. /Miss.
..... Reg. No..... of Third Year B.Sc., in
the Department of Computer Science, Shift-I, Tagore College of Arts & Science,
Chrompet, Chennai – 600 044 during the year 2026-2027

LECTURER IN – CHARGE

HEAD OF THE DEPARTMENT

Submitted for B.Sc., Degree Practical Examination held on
at the Department of Computer Science, Shift-I, Tagore College of Arts and
Science, Chrompet, Chennai –44.

Date :

Examiners

1.

2.

INDEX

S. No	DATE	EXERCISE	Page No	Sign
1.		DDL COMMENTS WITH CONSTRAINTS		
2.		DML COMMENTS WITH CONSTRAINTS		
3.		QUERIES, SUBQUERIES & AGGREGATE FUNCTION		
4.		PL/SQL – EXECUTION HANDLING		
5.		PL/SQL – CURSOR		
6.		PL/SQL – TRIGGER		
7.		PL/SQL – PACKAGES		
8.		LIBRARY MANAGEMENT SYSTEM		
9.		STUDENT MARKSHEET PROCESSING		
10.		PAYROLL PROCESSING		

1) DDL COMMENTS WITH CONSTRAINTS

DDL commands:

- * CREATE
- * ALTER
- * DROP
- * RENAME
- * TRUNCATE

Queries:

CREATE TABLE

Q1 → create table student(stid number, sname varchar(20), score integer);

Q2 → desc student;

Q3 → create table stu(sid number primary key, sname varchar(20) not null, email varchar(30) unique, age decimal(10,2) check (age>=18));

Q4 → desc stu;

ALTER TABLE

Q5 → alter table student drop column score;

Q6 → alter table student add gender varchar(5);

Q7 → alter table stu modify age number;

RENAME TABLE

Q8 → rename student to stu10;

Q9 → desc stu10;

Q10 → alter table stu rename column email to emailid;

Q11 → desc stu;

TRUNCATE AND DROP

Q12 → truncate table stu;

Q13 → drop table stu;

Q14 → desc stu;

OUTPUT:

CREATE TABLE

O/P → Table created.

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>STUDENT</u>	<u>STID</u>	Number	-	-	-	-	✓	-	-
	<u>STNAME</u>	Varchar2	20	-	-	-	✓	-	-
	<u>SCORE</u>	Number	-	-	0	-	✓	-	-

O/P →

O/P → Table created.

O/P →

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>STU</u>	<u>SID</u>	Number	-	-	-	1	-	-	-
	<u>SNAME</u>	Varchar2	20	-	-	-	-	-	-
	<u>EMAIL</u>	Varchar2	30	-	-	-	✓	-	-
	<u>AGE</u>	Number	-	10	2	-	✓	-	-

ALTER TABLE

O/P →

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>STUD</u>	<u>STID</u>	Number	-	-	-	-	✓	-	-
	<u>STN</u>	Varcha	20	-	-	-	✓	-	-

O/P → Table altered.

O/P → Table altered.

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>STU</u>	<u>SID</u>	Number	-	-	-	1	-	-	-
	<u>SNAME</u>	Varchar2	20	-	-	-	-	-	-
	<u>EMAIL</u>	Varchar2	30	-	-	-	✓	-	-
	<u>AGE</u>	Number	-	-	-	-	✓	-	-

RENAME TABLE

O/P → Statement processed.

O/P →

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>STU10</u>	<u>STID</u>	Number	-	-	-	-	✓	-	-
	<u>STNAME</u>	Varchar2	20	-	-	-	✓	-	-

<u>GENDER</u>	Varchar2	5	-	-	-	✓	-	-
-------------------------------	----------	---	---	---	---	---	---	---

O/P →

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>STU</u>	<u>SID</u>	Number	-	-	-	1	-	-	-
	<u>SNAME</u>	Varchar2	20	-	-	-	-	-	-
	<u>EMAIL</u>	Varchar2	30	-	-	-	✓	-	-
	<u>AGE</u>	Number	-	-	-	-	✓	-	-

TRUNCATE AND DROP

O/P → Table truncated.

O/P → Table dropped.

O/P → Object to be described could not be found.

2) DML COMMENTS WITH CONSTRAINTS

DML commands:

- * INSERT
- * UPDATE
- * DELETE
- * SELECT

Queries:

INSERT QUERY

Q1 → insert into stu10(stdid,sname,gender) values (1001,'aaa','f');

Q2 → insert into stu10(stdid,sname,gender) values (1002,'bbb','f');

Q3 → insert into stu10(stdid,sname,gender) values (1003,'cccc','f');

Q4 → insert into stu10(stdid,sname,gender) values (1004,'ddd','m');

Q5 → insert into stu10(stdid,sname,gender) values (1005,'EEE','M');

SELECT QUERY

Q6 → select * from stu10;

Q7 → select sname from stu10;

Q8 → select * from stu10 where gender='f';

UPDATE QUERY

Q9 → update stu10 set sname='anu' where stdid=1001;

Q10 → update stu10 set stname='bala', gender='m' where std=1002;

Q11 → select * from stu10;

DELETE QUERY

Q12 → delete from stu10 where std=1003;

Q13 → select * from stu10;

Q14 → delete from stu10;

Q15 → select * from stu10;

OUTPUT:

INSERT QUERY

O/P → 1 row(s) inserted.

SELECT QUERY

O/P

→

STID	STNAME	GENDER
1001	aaa	f
1002	bbb	f
1003	cccc	f
1004	ddd	m
1005	EEE	M

O/P →

STNAME
aaa
bbb
cccc
ddd
EEE

O/P →

STID	STNAME	GENDER
1001	aaa	f

1002	bbb	f
1003	cccc	f

UPDATE QUERY

O/P → 1 row(s) updated.

O/P → 1 row(s) updated.

O/P →

STID	STNAME	GENDER
1001	anu	f
1002	bala	m
1003	cccc	f
1004	ddd	m
1005	EEE	M

DELETE QUERY

O/P → 1 row(s) deleted.

O/P →

STID	STNAME	GENDER
1001	anu	f
1002	bala	m
1004	ddd	m

1005	EEE	M
------	-----	---

O/P → 4 row(s) deleted.

O/P → no data found

3) QUERIES, SUBQUERIES AND AGGREGATE FUNCTION

Queries:

BASIC QUERY

Q1 → create table empl(eid number primary key, ename varchar(20), deptid number, deptname varchar(20), salary number);

Q2 → insert into empl(eid,ename,deptid,deptname,salary) values (1201,'Arun',104,'CS',50000);

Q3 → insert into empl(eid,ename,deptid,deptname,salary) values (1202,'Bala',104,'CS',60000);

Q4 → insert into empl(eid,ename,deptid,deptname,salary) values (1203,'Chandru',105,'COMMERCE',40000);

Q5 → insert into empl(eid,ename,deptid,deptname,salary) values (1204,'Dilip',104,'CS',45000);

Q6 → insert into empl(eid,ename,deptid,deptname,salary) values (1205,'Dhanu',106,'TAMIL',30000);

Q7 → insert into empl(eid,ename,deptid,deptname,salary) values (1206,'Guru',106,'TAMIL',58000);

Q8 → insert into empl(eid,ename,deptid,deptname,salary) values (1207,'Hari',105,'COMMERCE',38000);

SELECT QUERY

Q9 → select * from empl;

Q10 → select eid,ename,salary from empl;

Q11 → select * from empl where salary>=50000;

AGGREGATE FUNCTION

Q12 → select COUNT(*) as totalemp from empl;

Q13 → select SUM(salary) as totalsal from empl;

Q14 → select AVG(salary) as totalsal from empl;

Q15 →select MAX(salary) as maxsalary from empl;

Q16 →select MIN(salary) as minsalary from empl;

Q17 →select eid,salary from empl order by salary desc;

Q18 →select deptid, sum(salary)as sal from empl group by deptid;

SUBQUERIES

Q19 → select * from empl where salary > (select avg(salary) from empl);

Q20 →create table dept(deptid number,deptname varchar(20));

Q21 →insert into dept(deptid,deptname) values (104,'CS');

Q22 → insert into dept(deptid,deptname) values (105,'COMMERCE');

Q23 → insert into dept(deptid,deptname) values (106,'TAMIL');

Q24 → insert into dept(deptid,deptname) values (107,'ENGLISH');

Q25 → select * from dept;

--- List department wise employee count:

Q26 → select d.deptname, e.empcount from dept d JOIN (select deptid, count(*) as empcount from empl group by deptid) e on d.deptid = e.deptid;

JOIN QUERIES

Q27 → select e.eid, e.ename, e.salary, d.deptid, d.deptname from empl e join dept d on e.deptid = d.deptid;

Q28 → select e.eid, e.ename, e.salary, d.deptid, d.deptname from empl e LEFT JOIN dept d on e.deptid = d.deptid;

Q29 → select e.eid, e.ename, e.salary, d.deptid, d.deptname from empl e RIGHT JOIN dept d on e.deptid = d.deptid;

Q30 → select e.eid, e.ename, e.salary, d.deptid, d.deptname from empl e CROSS JOIN dept d;

OUTPUT:

BASIC QUERY

O/P → Table created.

SELECT QUERY

O/P

→

EID	ENAME	DEPTID	DEPTNAME	SALARY
1201	Arun	104	CS	50000
1202	Bala	104	CS	60000
1203	Chandru	105	COMMERCE	40000
1204	Dilip	104	CS	45000
1205	Dhanu	106	TAMIL	30000
1206	Guru	106	TAMIL	58000
1207	Hari	105	COMMERCE	38000

O/P →

EID	ENAME	SALARY
1201	Arun	50000
1202	Bala	60000
1203	Chandru	40000
1204	Dilip	45000
1205	Dhanu	30000
1206	Guru	58000
1207	Hari	38000

O/P →

EID	ENAME	DEPTID	DEPTNAME	SALARY
1201	Arun	104	CS	50000
1202	Bala	104	CS	60000
1206	Guru	106	TAMIL	58000

AGGREGATE FUNCTION

O/P →

TOTALEMP
7

O/P →

TOTALSAL
321000

O/P →

TOTALSAL
45857.1428571428571428571428571429

O/P →

MAXSALARY
60000

O/P

→

MINSALARY
30000

O/P

→EID	SALARY
1202	60000
1206	58000
1201	50000
1204	45000
1203	40000
1207	38000
1205	30000

O/P →

DEPTID	SAL
104	155000
105	78000
106	88000

SUBQUERIES

O/P →

EID	ENAME	DEPTID	DEPTNAME	SALARY
------------	--------------	---------------	-----------------	---------------

1201	Arun	104	CS	50000
1202	Bala	104	CS	60000
1206	Guru	106	TAMIL	58000

O/P →

DEPTID	DEPTNAME
104	CS
105	COMMERCE
106	TAMIL
107	ENGLISH

O/P →

DEPTNAME	EMPCOUNT
COMMERCE	2
CS	3
TAMIL	2

JOIN QUERIES

O/P →

EID	ENAME	SALARY	DEPTID	DEPTNAME
1201	Arun	50000	104	CS
1202	Bala	60000	104	CS
1203	Chandru	40000	105	COMMERCE

1204	Dilip	45000	104	CS
1205	Dhanu	30000	106	TAMIL
1206	Guru	58000	106	TAMIL
1207	Hari	38000	105	COMMERCE

O/P →

EID	ENAME	SALARY	DEPTID	DEPTNAME
1204	Dilip	45000	104	CS
1202	Bala	60000	104	CS
1201	Arun	50000	104	CS
1207	Hari	38000	105	COMMERCE
1203	Chandru	40000	105	COMMERCE
1206	Guru	58000	106	TAMIL
1205	Dhanu	30000	106	TAMIL

O/P →

EID	ENAME	SALARY	DEPTID	DEPTNAME
1201	Arun	50000	104	CS
1202	Bala	60000	104	CS
1203	Chandru	40000	105	COMMERCE
1204	Dilip	45000	104	CS
1205	Dhanu	30000	106	TAMIL
1206	Guru	58000	106	TAMIL

1207	Hari	38000	105	COMMERCE
-	-	-	107	ENGLISH

O/P →

EID	ENAME	SALARY	DEPTID	DEPTNAME
1201	Arun	50000	104	CS
1202	Bala	60000	104	CS
1203	Chandru	40000	104	CS
1204	Dilip	45000	104	CS
1205	Dhanu	30000	104	CS
1206	Guru	58000	104	CS
1207	Hari	38000	104	CS
1201	Arun	50000	105	COMMERCE
1202	Bala	60000	105	COMMERCE
1203	Chandru	40000	105	COMMERCE

4) PL/SQL – EXCEPTION HANDLING

Queries:

Predefined Exceptions

Q1 →

DECLARE

num1 NUMBER := 10;

num2 NUMBER := 0;

result NUMBER;

BEGIN

result := num1 / num2;

DBMS_OUTPUT.PUT_LINE('Result: ' || result);

EXCEPTION

WHEN ZERO_DIVIDE THEN

DBMS_OUTPUT.PUT_LINE('Cannot divide by zero.');

END;

User-Defined Exceptions

Q2 →

DECLARE

insufficient_balance EXCEPTION;

balance NUMBER := 500;

BEGIN

IF balance < 1000 THEN

RAISE insufficient_balance;

END IF;

EXCEPTION

WHEN insufficient_balance THEN

```
DBMS_OUTPUT.PUT_LINE('Balance is too low.');
```

END;

Named System Exceptions

Q3 →

```
DECLARE

    ename emp.name%TYPE;

BEGIN

    SELECT name into ename

    FROM emp

    WHERE eid = 1001; -- assuming invalid ID

    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || ename);

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('No employee found with that ID.');
```

END;

Unhandled (OTHERS) Exception

Q4 →

```
BEGIN

    EXECUTE IMMEDIATE 'DROP TABLE nonexistent_table';

EXCEPTION

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Some unexpected error occurred: ' || SQLERRM);

END;
```

OUTPUT:

Predefined Exceptions

O/P

→

Cannot divide by zero.

User-Defined Exceptions

O/P

→

Balance is too low.

Named System Exceptions

O/P

→

No employee found with that ID.

Unhandled (OTHERS) Exception

O/P

→

Some unexpected error occurred: ORA-00942: table or view does not exist

5) PL/SQL – CURSOR

Queries:

Q1 → select * from emp;

Implicit Cursor

Q2 →

BEGIN

UPDATE emp SET salary = salary + 1000 WHERE eid = 103;

IF SQL%ROWCOUNT = 1 THEN

DBMS_OUTPUT.PUT_LINE('1 row updated successfully.');

END IF;

END;

Q3 → select * from emp;

Explicit Cursor

Q4 →

DECLARE

CURSOR emp_cursor IS SELECT eid, name FROM emp;

v_id emp.eid%TYPE;

v_name emp.name%TYPE;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor INTO v_id, v_name;

EXIT WHEN emp_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE('ID: ' || v_id || ', Name: ' || v_name);

```
END LOOP;  
CLOSE emp_cursor;  
END;
```

Cursor FOR Loop

Q5 →

```
BEGIN  
FOR emp_rec IN (SELECT eid, name FROM emp) LOOP  
  DBMS_OUTPUT.PUT_LINE('ID: ' || emp_rec.eid || ', Name: ' || emp_rec.name);  
END LOOP;  
END;
```

Parameterized Cursor

Q6 →

```
DECLARE  
  
CURSOR emp_cursor(p_dept_id NUMBER) IS  
  SELECT eid, name FROM emp WHERE dept_id = p_dept_id;  
  v_id emp.eid%TYPE;  
  v_name emp.name%TYPE;  
  
BEGIN  
  OPEN emp_cursor(123); -- passing department id  
  
  LOOP  
    FETCH emp_cursor INTO v_id, v_name;  
    EXIT WHEN emp_cursor%NOTFOUND;  
    DBMS_OUTPUT.PUT_LINE('ID: ' || v_id || ', Name: ' || v_name);  
  END LOOP;  
  
  CLOSE emp_cursor;  
END;
```

OUTPUT:

O/P

→

EID	NAME	DOJ	SALARY	DEPT_ID
103	Cholan	12-FEB-20	30000	123
104	Hari	22-FEB-24	20000	122
101	Arun	12-FEB-20	40000	123
102	Bala	01-DEC-19	50000	121
105	Guru	10-JAN-18	60000	122
106	Mathi	01-JUN-20	46000	121

Implicit Cursor

O/P →

1 row updated successfully.

1 row(s) updated.

O/P →

EID	NAME	DOJ	SALARY	DEPT_ID
103	Cholan	12-FEB-20	31000	123
104	Hari	22-FEB-24	20000	122
101	Arun	12-FEB-20	40000	123
102	Bala	01-DEC-19	50000	121
105	Guru	10-JAN-18	60000	122
106	Mathi	01-JUN-20	46000	121

Explicit Cursor

O/P

→

ID: 103, Name: Cholan

ID: 104, Name: Hari

ID: 101, Name: Arun

ID: 102, Name: Bala

ID: 105, Name: Guru

ID: 106, Name: Mathi

Cursor FOR Loop

O/P

→

ID: 103, Name: Cholan

ID: 104, Name: Hari

ID: 101, Name: Arun

ID: 102, Name: Bala

ID: 105, Name: Guru

ID: 106, Name: Mathi

Parameterized Cursor

O/P

→

ID: 103, Name: Cholan

ID: 101, Name: Arun

6) PL/SQL – TRIGGER

PL/SQL Queries:

Q1 → CREATE TABLE students (roll_no NUMBER PRIMARY KEY,
name VARCHAR2(50), marks NUMBER);

BEFORE INSERT TRIGGER

Q2 →
CREATE OR REPLACE TRIGGER trg_before_insert_students
BEFORE INSERT ON students
FOR EACH ROW
BEGIN
-- If marks not given, set to 0
IF :NEW.marks IS NULL THEN
:NEW.marks := 0;
END IF;
END;

AFTER INSERT TRIGGER

Q3 →
CREATE OR REPLACE TRIGGER trg_after_insert_students
AFTER INSERT ON students
BEGIN
DBMS_OUTPUT.PUT_LINE('A new student was added!');
END;

Q4 → INSERT INTO students (roll_no, name) VALUES (1, 'John');

Q5 → select * from students;

BEFORE UPDATE TRIGGER

Q6 →

CREATE OR REPLACE TRIGGER trg_before_update_students

BEFORE UPDATE ON students

FOR EACH ROW

BEGIN

-- Prevent marks from being more than 100

IF :NEW.marks > 100 THEN

:NEW.marks := 100;

END IF;

END;

Q7 → UPDATE students SET marks = 120 WHERE roll_no = 1;

Q8 → select * from students;

AFTER DELETE TRIGGER

Q9 →

CREATE OR REPLACE TRIGGER trg_after_delete_students

AFTER DELETE ON students

BEGIN

DBMS_OUTPUT.PUT_LINE('A student record was deleted!');

END;

Q10 → DELETE FROM students WHERE roll_no = 1;

OUTPUT:

O/P →

Table Created.

BEFORE INSERT TRIGGER

O/P →

Trigger created.

AFTER INSERT TRIGGER

O/P →

Trigger created.

O/P →

A new student was added!

1 row(s) inserted.

O/P

→

ROLL_NO	NAME	MARKS
1	John	0

BEFORE UPDATE TRIGGER

O/P →

Trigger created.

O/P →

1 row(s) updated.

O/P

→

ROLL_NO	NAME	MARKS
1	John	100

AFTER DELETE TRIGGER

O/P → Trigger created.

O/P → A student record was deleted!

1 row(s) deleted.

7) PL/SQL – PACKAGES

PL/SQL Queries:

Package Specification

Q1 →

```
CREATE OR REPLACE PACKAGE math_pkg AS  
    FUNCTION square_num(p_num NUMBER) RETURN NUMBER;  
END math_pkg;
```

Package Body

Q2 →

```
CREATE OR REPLACE PACKAGE BODY math_pkg AS  
    FUNCTION square_num(p_num NUMBER) RETURN NUMBER IS  
    BEGIN  
        RETURN p_num * p_num;  
    END;  
END math_pkg;
```

Test the Package

Q3 →

```
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Square of 5: ' || math_pkg.square_num(5));  
END;
```

Predefined Package

Q4 →

```
BEGIN  
    -- Random number between 1 and 50  
    DBMS_OUTPUT.PUT_LINE ('Random Number: ' || ROUND(DBMS_RANDOM.VALUE(1, 50)));  
  
    -- Random uppercase string of length 8  
    DBMS_OUTPUT.PUT_LINE('Random String: ' || DBMS_RANDOM.STRING('U', 8));  
END;
```

OUTPUT:

Package Specification

O/P →

Package created.

Package Body

O/P →

Package Body created.

Test the Package

O/P →

Square of 5: 25

Predefined Package

O/P

→

Random Number: 34

Random String: ZOWXBLHK

8) LIBRARY MANAGEMENT SYSTEM

Coding:

SQL:

```
create table book ( bookid number primary key,  
    bookname varchar2(30) ,  
    author varchar2(20) ,  
    quan number  
);
```

Form1 (VB 2010):

```
Imports System.Data.OleDb
```

```
Public Class Form1
```

```
    ' Form-level connection to Oracle 10g
```

```
    Dim cn As New OleDbConnection("Provider=MSDAORA;Data Source=XE;User  
ID=system;Password=system;")
```

```
    ' --- 1. ADD BOOK ---
```

```
    Private Sub Button1_Click(ByVal sender As Object, ByVal e As EventArgs) Handles Button1.Click
```

```
        ' 1. Check if any of your textboxes are empty first
```

```
        If TextBox1.Text = "" Or TextBox2.Text = "" Or TextBox3.Text = "" Or TextBox4.Text = "" Then
```

```
            MsgBox("Please fill in all fields before adding a book.")
```

```
            Exit Sub ' Stops the code here so Oracle doesn't crash
```

```
        End If
```

```
    Try
```

```
        cn.Open()
```

```
        Dim sql As String = "INSERT INTO book VALUES (" & TextBox1.Text & ", " & TextBox2.Text  
            & ", " & TextBox3.Text & ", " & TextBox4.Text & ")"
```

```
        Dim cmd As New OleDbCommand(sql, cn)
```

```
        cmd.ExecuteNonQuery()
```

```
        MsgBox("Book Added Successfully!")
```

```
        TextBox1.Clear()
```

```
        TextBox2.Clear()
```

```
        TextBox3.Clear()
```

```
        TextBox4.Clear()
```

```
    Catch ex As Exception
```

```
        MsgBox("Error adding book: " & ex.Message)
```

```
    Finally
```

```
        cn.Close()
```

```
    End Try
```

```
End Sub
```

' --- 2. ISSUE BOOK ---

Private Sub Button2_Click(ByVal sender As Object, ByVal e As EventArgs) Handles Button2.Click

' We use InputBox here so you don't have to fill out the textboxes just to issue a book

Dim bid As String = InputBox("Enter the Book ID to Issue:")

If bid = "" Then Exit Sub

Try

cn.Open()

Dim sql As String = "UPDATE book SET quan = quan - 1 WHERE bookid = " & bid

Dim cmd As New OleDbCommand(sql, cn)

Dim rowsAffected As Integer = cmd.ExecuteNonQuery()

If rowsAffected > 0 Then

MsgBox("Book Issued Successfully!")

Else

MsgBox("No record found with ID: " & bid)

End If

Catch ex As Exception

MsgBox("Error issuing book: " & ex.Message)

Finally

cn.Close()

End Try

End Sub

' --- 3. RETURN BOOK ---

Private Sub Button3_Click(ByVal sender As Object, ByVal e As EventArgs) Handles Button3.Click

Dim bid As String = InputBox("Enter the Book ID to Return:")

If bid = "" Then Exit Sub

Try

cn.Open()

Dim sql As String = "UPDATE book SET quan = quan + 1 WHERE bookid = " & bid

Dim cmd As New OleDbCommand(sql, cn)

Dim rowsAffected As Integer = cmd.ExecuteNonQuery()

If rowsAffected > 0 Then

MsgBox("Book Returned Successfully!")

Else

MsgBox("No record found with ID: " & bid)

End If

Catch ex As Exception

MsgBox("Error returning book: " & ex.Message)

Finally

cn.Close()

End Try

End Sub

' --- 4. VIEW REPORT (Updates the DataGridView at the bottom) ---

Private Sub Button4_Click(ByVal sender As Object, ByVal e As EventArgs) Handles Button4.Click

Try

Dim sql As String = "SELECT * FROM book"

' DataAdapter grabs the data without needing to manually open/close the connection

Dim da As New OleDbDataAdapter(sql, cn)

Dim dt As New DataTable()

da.Fill(dt)

DataGridView1.DataSource = dt

Catch ex As Exception

MsgBox("Error loading data: " & ex.Message)

End Try

End Sub

End Class

Output:

test_main

lib management

bookID author

book name quan

	BOOKID	BOOKNAME	AUTHOR	QUAN
▶	101	RDBMS	Severus	55
*				

test_main

lib management

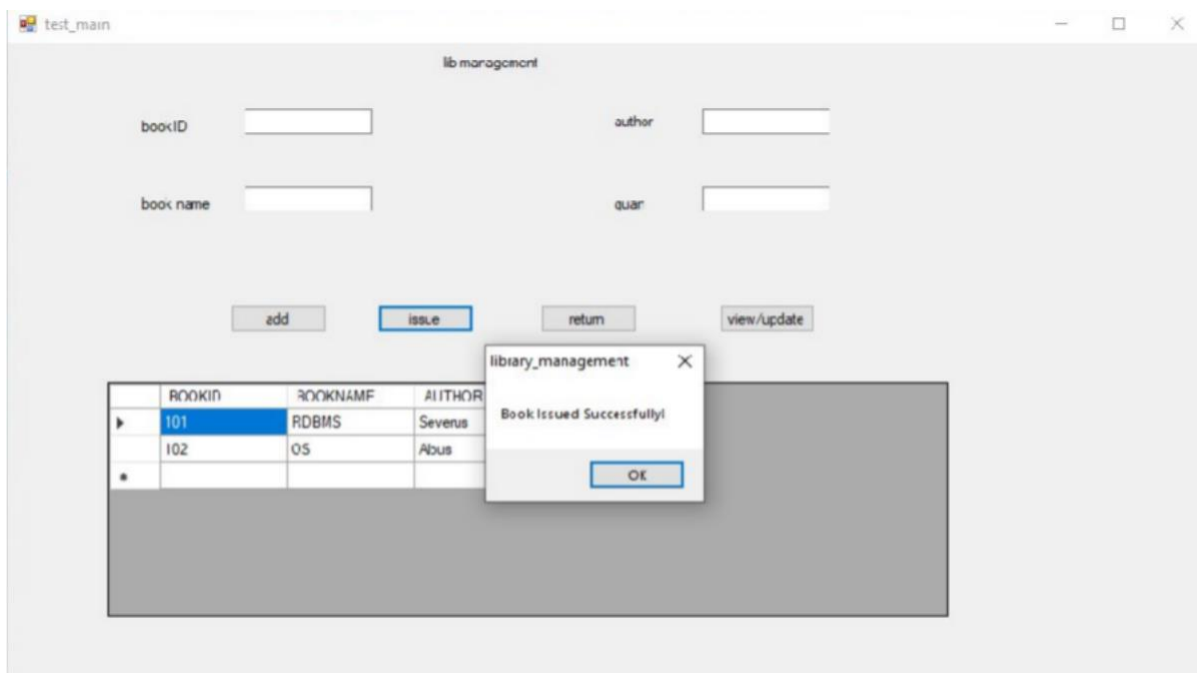
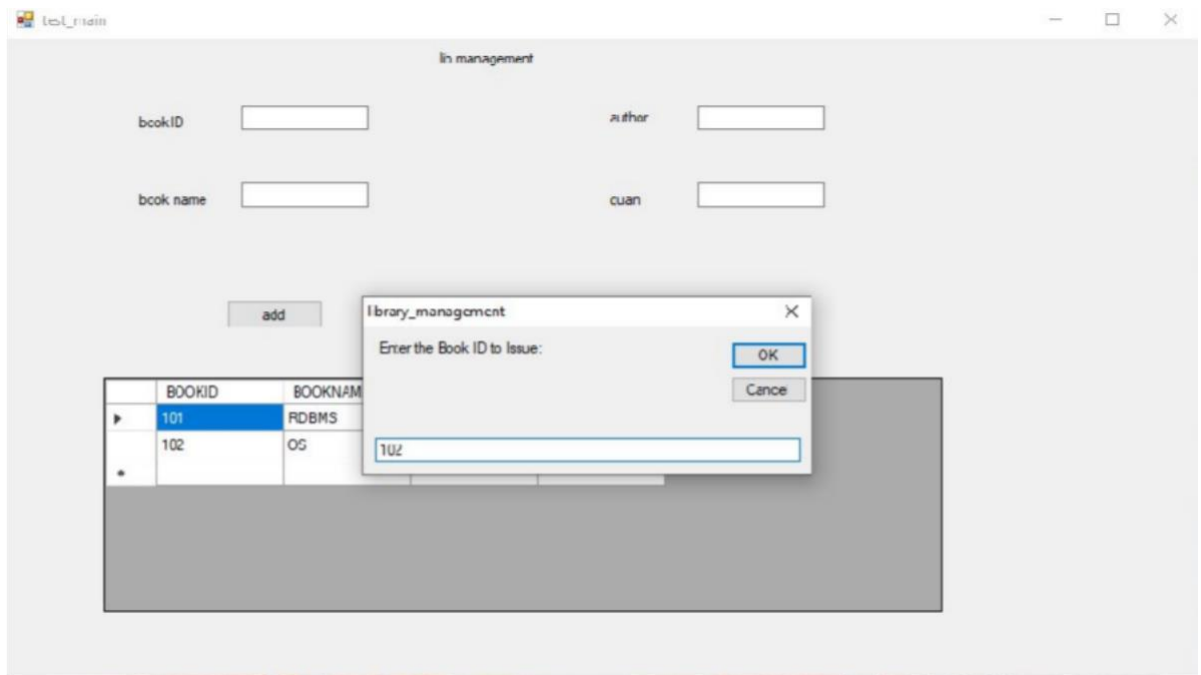
bookID author

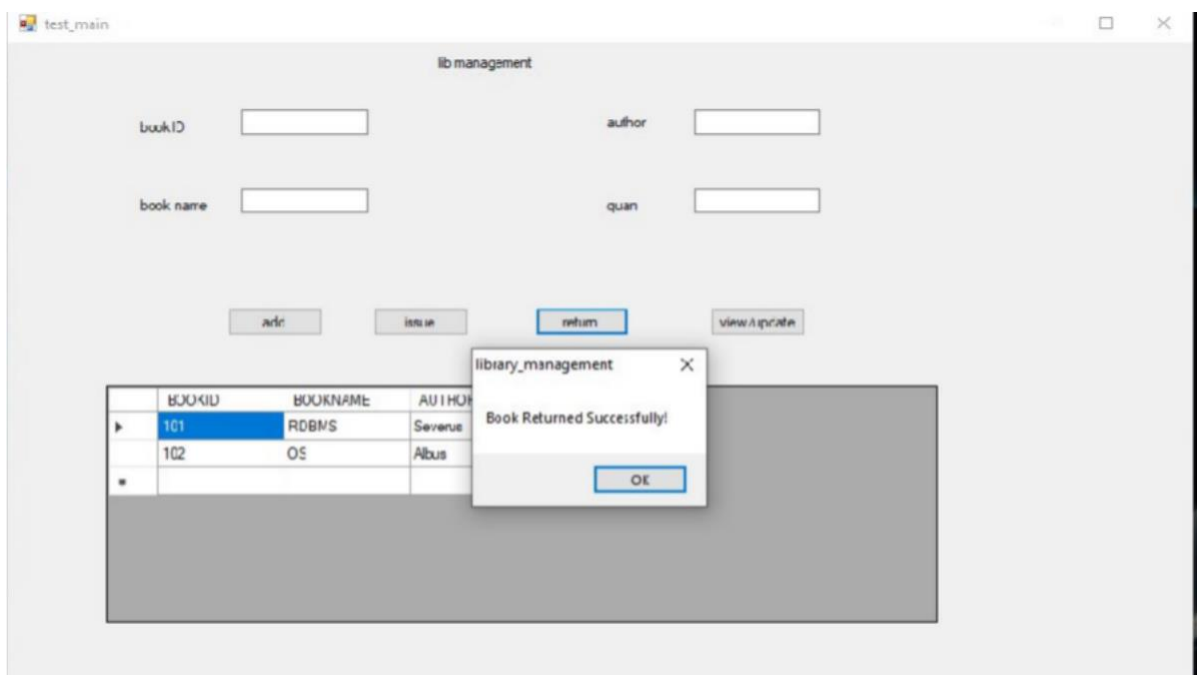
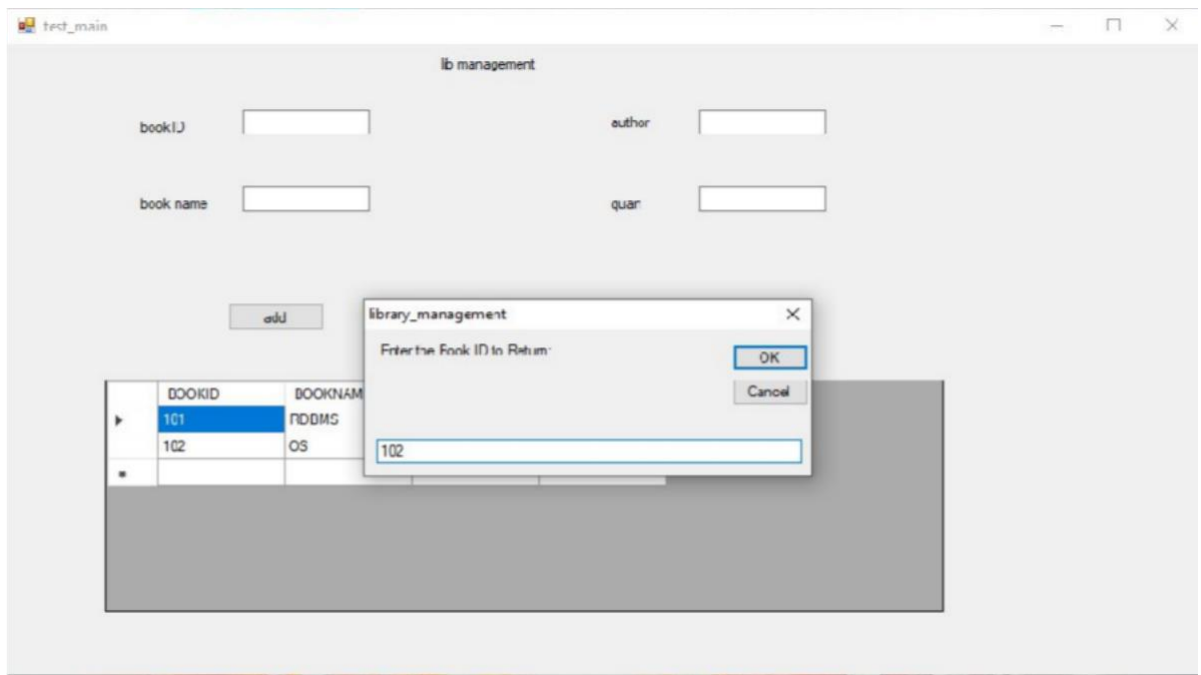
book name quan

	BOOKID	BOOKNAME	AUTHOR
▶	101	RDBMS	Severus
*			

library_management

Book Added Successfully!





9) STUDENT MARKSHEET PROCESSING

Coding:

SQL:

```
CREATE TABLE st (  
    stid NUMBER PRIMARY KEY,  
    stname VARCHAR2(50),  
    os NUMBER,  
    rdbms NUMBER,  
    cn NUMBER,  
    se NUMBER,  
    total NUMBER,  
    result VARCHAR2(10),  
    percent NUMBER  
);
```

Form1 :

Imports System.Data.OleDb

Public Class Form1

```
    Dim cn As New OleDbConnection("Provider=MSDAORA;Data Source=XE;User  
ID=system;Password=system;")
```

```
    ' --- CALCULATE ---
```

```
    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click  
        TextBox7.Text = (Val(TextBox3.Text) + Val(TextBox4.Text) + Val(TextBox5.Text) +  
Val(TextBox6.Text)).ToString()
```

```
        If Val(TextBox3.Text) <= 30 Or Val(TextBox4.Text) <= 30 Or Val(TextBox5.Text) <= 30  
Or Val(TextBox6.Text) <= 30 Then
```

```
            TextBox8.Text = "FAIL"
```

```
        Else
```

```
            TextBox8.Text = "PASS"
```

```
        End If
```

```
        TextBox9.Text = (Val(TextBox7.Text) / 4).ToString()
```

```
    End Sub
```

' --- INSERT ---

Private Sub Button2_Click(sender As Object, e As EventArgs) Handles Button2.Click

Button1.PerformClick() ' auto-calculate before insert

Try

cn.Open()

Dim sql As String = "INSERT INTO st VALUES (" & TextBox1.Text & ", " & TextBox2.Text & ", " & TextBox3.Text & ", " & TextBox4.Text & ", " & TextBox5.Text & ", " & TextBox6.Text & ", " & TextBox7.Text & ", " & TextBox8.Text & ", " & TextBox9.Text & ")"

Dim cmd As New OleDbCommand(sql, cn)

cmd.ExecuteNonQuery()

MsgBox("Successfully Inserted")

TextBox1.Clear(): TextBox2.Clear(): TextBox3.Clear()

TextBox4.Clear(): TextBox5.Clear(): TextBox6.Clear()

TextBox7.Clear(): TextBox8.Clear(): TextBox9.Clear()

Catch ex As Exception

MsgBox(ex.Message)

Finally

cn.Close()

End Try

End Sub

' --- DELETE ---

Private Sub Button3_Click(sender As Object, e As EventArgs) Handles Button3.Click

Dim studentID As String = InputBox("Enter the Student ID to delete:")

If studentID = "" Then Exit Sub

Try

cn.Open()

Dim cmd As New OleDbCommand("DELETE FROM st WHERE stid = " & studentID, cn)

Dim rowsAffected As Integer = cmd.ExecuteNonQuery()

If rowsAffected > 0 Then

MsgBox("Record with ID " & studentID & " deleted successfully!")

Else

MsgBox("No record found with ID: " & studentID)

End If

```
Catch ex As Exception
    MsgBox(ex.Message)
Finally
    cn.Close()
End Try
End Sub

' --- VIEW ---
Private Sub Button4_Click(sender As Object, e As EventArgs) Handles Button4.Click
    Try
        Dim da As New OleDbDataAdapter("SELECT * FROM st", cn)
        Dim dt As New DataTable()
        da.Fill(dt)
        DataGridView1.DataSource = dt
    Catch ex As Exception
        MsgBox(ex.Message)
    End Try
End Sub
End Class
```

Output:

STUDENT MARKSHEET PROCESSING

STUDENT ID	12345	CALCULATE	
STUDENT NAME	Madhu	TOTAL MARKS	324
OS	67	RESULT	PASS
RDBMS	89	PERCENTAGE	81
CN	90		
SE	78		

REPORT INSERT DELETE

student_idb
Successfully Inserted
OK

STUDENT MARKSHEET PROCESSING

STUDENT ID		CALCULATE	
STUDENT NAME		TOTAL MARKS	
OS		RESULT	
RDBMS		PERCENTAGE	
CN			
SE			

REPORT INSERT DELETE

student_idb
Enter the Student ID to delete:
234
OK Cancel

Forest

STUDENT MARKSHEET PROCESSING

STUDENT ID **CALCULATE**

STUDENT NAME **TOTAL MARKS**

OS **RESULT**

RDBMS **PERCENTAGE**

CN

SE

REPORT **INSERT** **DELETE**

student_db

Record with ID 234 deleted successfully!

OK

Barry days ahead 29°C Search 10:41 26-09-2023

DataReport1

Zoom 100%

STUDENT ID	stname:	result:	percent:
67	dyh	Pass	68.75
34	dyty	Pass	68.75
4545	tyu	Pass	68.75

Page: 1/1

Barry days ahead 29°C Search 10:41 26-09-2023

10) PAYROLL PROCESSING

Coding:

SQL:

```
CREATE TABLE emp (  
    empid NUMBER PRIMARY KEY,  
    empname VARCHAR2(50),  
    salary NUMBER,  
    gross NUMBER,  
    net NUMBER );
```

Form1 :

Imports System.Data.OleDb

Public Class Form1

Dim cn As New OleDbConnection("Provider=MSDAORA;Data Source=XE;User ID=system;
Password=system;") `if data source error occurs use orcl instead of XE

' --- CALCULATE ---

Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click

TextBox4.Text = (Val(TextBox3.Text) * 0.25).ToString() ' Allowance

TextBox5.Text = (Val(TextBox3.Text) * 0.12).ToString() ' Deduction

TextBox6.Text = (Val(TextBox3.Text) + Val(TextBox4.Text)).ToString() ' Gross

TextBox7.Text = (Val(TextBox6.Text) - Val(TextBox5.Text)).ToString() ' Net

End Sub

' --- INSERT ---

Private Sub Button2_Click(sender As Object, e As EventArgs) Handles Button2.Click

Button1.PerformClick() ' auto-calculate before insert

Try

cn.Open()

Dim sql As String = "INSERT INTO emp VALUES (" & TextBox1.Text & ", " &
TextBox2.Text & ", " & TextBox3.Text & ", " & TextBox6.Text & ", " & TextBox7.Text & ")"

Dim cmd As New OleDbCommand(sql, cn)

cmd.ExecuteNonQuery()

MsgBox("Successfully Inserted")

TextBox1.Clear(): TextBox2.Clear(): TextBox3.Clear()

TextBox4.Clear(): TextBox5.Clear(): TextBox6.Clear()

TextBox7.Clear()

Catch ex As Exception

MsgBox(ex.Message)

Finally

cn.Close()

End Try

End Sub

' --- DELETE ---

Private Sub Button3_Click(sender As Object, e As EventArgs) Handles Button3.Click

Dim eid As String = InputBox("Enter the EMPLOYEE ID to delete:")

If eid = "" Then Exit Sub

Try

cn.Open()

Dim cmd As New OleDbCommand("DELETE FROM emp WHERE empid = " & eid, cn)

Dim rowsAffected As Integer = cmd.ExecuteNonQuery()

If rowsAffected > 0 Then

MsgBox("Record with ID " & eid & " deleted successfully!")

Else

MsgBox("No record found with ID: " & eid)

End If

Catch ex As Exception

MsgBox(ex.Message)

Finally

cn.Close()

End Try

End Sub

' --- VIEW ---

Private Sub Button4_Click(sender As Object, e As EventArgs) Handles Button4.Click

Try

Dim da As New OleDbDataAdapter("SELECT * FROM emp", cn)

Dim dt As New DataTable()

da.Fill(dt)

DataGridView1.DataSource = dt

Catch ex As Exception

MsgBox(ex.Message)

End Try

End Sub

' --- EXIT ---

Private Sub Button5_Click(sender As Object, e As EventArgs) Handles Button5.Click

Application.Exit()

End Sub

End Class

Output:

EMPLOYEE PAYROLL SYSTEM

EMPLOYEE ID	1001
EMPLOYEE NAME	Amelia
SALARY	9000
CALCULATE	
TOTAL ALLOWANCE	2000
TOTAL DEDUCTION	900
GROSS SALARY	10000
<i>NET SALARY</i>	90000

INSERT
DELETE
DATA REPORT
EXIT

EMPROLL
Successfully Inserted
OK

EMPLOYEE PAYROLL SYSTEM

EMPLOYEE ID	
EMPLOYEE NAME	
SALARY	
CALCULATE	
TOTAL ALLOWANCE	
TOTAL DEDUCTION	
GROSS SALARY	
<i>NET SALARY</i>	

INSERT
DELETE
DATA REPORT
EXIT

EMPROLL
Enter the EMPLOYEE ID to delete
OK
Cancel
1001

